

AulaVirtual Pro

LMS para Capacitación Empresarial

DOCUMENTO SCRUM — MVP

Épicas · Historias de Usuario · Tareas · Estimación de Esfuerzo

Proyecto	AulaVirtual Pro — Sistema LMS Empresarial
Cliente	CapacitaTec S.A.
Equipo	Equipo Alfa
Stack	React · Node.js · Express · MongoDB Atlas · JWT · Cloudinary · Nodemailer
Metodología	Scrum — Sprints de 1 semana
Año	2026

Integrantes del Equipo

Integrante	Rol Scrum	Responsabilidad técnica
Alberto Anyelo	Scrum Master / DevOps	Gestión del proyecto, MongoDB Atlas, GitHub, CI/CD, diagramas
Erick Hernandez	Developer — Backend	API REST Node.js/Express, JWT, modelos MongoDB, Nodemailer
Johan Anuart	Developer — Frontend	React, React Router, Axios, AuthContext, páginas y componentes
Alan Hazel	UX/UI Designer	Figma, Design System, componentes React, QA visual, documentación

1. Visión del Producto y Criterios de Aceptación

Visión: AulaVirtual Pro es un sistema LMS (Learning Management System) que permite a CapacitaTec S.A. digitalizar su proceso de capacitación empresarial, reemplazando la distribución de materiales por WhatsApp por una plataforma estructurada con cursos, evaluaciones y seguimiento de progreso por alumno.

Criterios de aceptación del MVP

ID	Criterio	Estado MVP
CA-01	Un alumno puede registrarse e iniciar sesión con JWT	Implementado ✔
CA-02	El sistema distingue 3 roles: admin, instructor, alumno	Implementado ✔
CA-03	Un instructor puede crear cursos con módulos y lecciones	Implementado ✔
CA-04	Un alumno puede inscribirse a cursos disponibles	Implementado ✔
CA-05	El progreso por lección se guarda y muestra en porcentaje	Implementado ✔
CA-06	El alumno puede realizar un quiz con calificación automática	Implementado ✔
CA-07	El instructor puede crear y asignar tareas por curso	Implementado ✔
CA-08	El admin puede gestionar usuarios (CRUD) desde su panel	Implementado ✔
CA-09	El sistema envía emails de bienvenida e inscripción (Nodemailer)	Implementado ✔
CA-10	El admin puede ver reportes de progreso y calificaciones	Implementado ✔

2. Épicas del Proyecto

Una épica es un bloque grande de funcionalidad que agrupa múltiples historias de usuario.
AulaVirtual Pro se divide en 6 épicas que cubren el alcance completo del MVP.

ID	Épica	Descripción	Responsable	Story Points
EP-01	Autenticación y Roles	Registro, login con JWT, control de acceso por rol (admin/instructor/alumno), middleware de seguridad	Erick	21
EP-02	Gestión de Cursos	CRUD de cursos, módulos, lecciones y materiales adjuntos por URL. Vista alumno e instructor	Erick + Johan	34
EP-03	Evaluaciones	Creación de quizzes con preguntas de opción múltiple, calificación automática y visualización de resultados	Erick + Johan	21
EP-04	Inscripciones y Progreso	Inscripción a cursos, barra de progreso por lección, dashboard del alumno con estadísticas	Erick + Johan	21
EP-05	Gestión Administrativa	Panel admin con CRUD de usuarios, reportes de progreso, tareas asignadas por instructor	Erick + Johan	21
EP-06	Notificaciones e Integración	Envío de emails con Nodemailer (bienvenida, inscripción). Preparación para Cloudinary	Erick	13
TOTAL				131 pts

3. Historias de Usuario y Criterios

EP-01 — Autenticación y Roles

US-01	Registro de alumno	5 SP pts
Como...	alumno nuevo	
Quiero...	registrarme con nombre, email y contraseña	
Para...	acceder a la plataforma con mi cuenta	

Criterios de aceptación:

- ✓ El formulario valida que los 3 campos estén completos
- ✓ La contraseña se encripta con bcryptjs antes de guardarse
- ✓ Se genera y retorna un token JWT al completar el registro
- ✓ Se envía un email de bienvenida automáticamente
- ✓ Si el email ya existe, el sistema retorna error 409

Tareas técnicas:

Capa	Tarea	Responsable	Est.
BE	Crear modelo User con nombre, email, password, rol, activo	Erick	2h
BE	Implementar POST /api/v1/auth/register con validaciones	Erick	2h
BE	Integrar bcryptjs para hashear password	Erick	1h
BE	Generar token JWT con payload {id, rol}	Erick	1h
BE	Enviar email de bienvenida con Nodemailer	Erick	1h
FE	Crear RegisterPage con formulario de 3 campos	Johan	2h
FE	Conectar formulario a POST /auth/register con Axios	Johan	1h

US-02	Inicio de sesión	5 SP pts
Como...	usuario registrado	
Quiero...	iniciar sesión con email y contraseña	
Para...	acceder a las funciones según mi rol	

Criterios de aceptación:

- ✓ El login valida credenciales y retorna JWT con {id, rol}
- ✓ El token se guarda en localStorage y se adjunta en cada request
- ✓ Si las credenciales son incorrectas, retorna error 401
- ✓ La redirección tras login depende del rol: /catalog, /teacher o /admin

Tareas técnicas:

Capa	Tarea	Responsable	Est.
BE	Implementar POST /api/v1/auth/login con bcrypt.compare()	Erick	2h
BE	Crear middleware verifyToken para rutas protegidas	Erick	2h
BE	Crear middleware checkRole para control por rol	Erick	1h
FE	Crear LoginPage con validación de campos	Johan	2h
FE	Implementar AuthContext con estado global de sesión	Johan	2h
FE	Configurar interceptor Axios para adjuntar JWT	Johan	1h
FE	Implementar PrivateRoute y RoleRoute para rutas protegidas	Johan	2h

US-03	Cierre de sesión	2 SP pts
Como...	usuario autenticado	
Quiero...	cerrar sesión desde cualquier pantalla	
Para...	proteger mi cuenta al terminar	

Criterios de aceptación:

- ✓ El botón de logout limpia el token de localStorage
- ✓ Redirige automáticamente a /login
- ✓ Las rutas protegidas ya no son accesibles sin token

Tareas técnicas:

Capa	Tarea	Responsable	Est.
FE	Implementar logout() en AuthContext que limpia localStorage	Johan	1h
FE	Agregar botón de cierre de sesión en todos los dashboards	Johan	1h

US-04	Control de acceso por rol	5 SP pts
Como...	sistema	
Quiero...	verificar el rol antes de acceder a cada endpoint y pantalla	
Para...	que cada actor solo acceda a lo que le corresponde	

Criterios de aceptación:

- ✓ Sin token: respuesta 401 en todos los endpoints protegidos
- ✓ Con token de rol incorrecto: respuesta 403 Forbidden
- ✓ Admin puede acceder a /admin, instructor a /teacher, alumno a /catalog
- ✓ Rutas de React redirigen al dashboard del rol correspondiente

Tareas técnicas:

Capa	Tarea	Responsable	Est.
BE	Aplicar verifyToken + checkRole en todas las rutas protegidas	Erick	2h
FE	Implementar lógica de redirección por rol en App.jsx	Johan	2h

EP-02 — Gestión de Cursos

US-05	Crear y editar cursos	8 SP pts
Como...	instructor	
Quiero...	crear y editar cursos con título, descripción e imagen	
Para...	ofrecer contenido organizado a los alumnos	

Criterios de aceptación:

- ✓ Solo instructor o admin pueden crear/editar cursos
- ✓ Un instructor solo puede editar sus propios cursos
- ✓ El curso tiene título obligatorio, descripción e imagen opcional
- ✓ CRUD completo: crear, leer, actualizar, desactivar (soft delete)

Tareas técnicas:

Capa	Tarea	Responsable	Est.
BE	Crear modelo Course con modulos[] embebidos e índices	Erick	2h
BE	Implementar CRUD /api/v1/cursos con checkRole	Erick	3h
FE	Crear TeacherDashboard con vista lista, crear y editar	Johan	4h
FE	Botón eliminar con confirmación y actualización de lista	Johan	1h

US-06	Gestionar módulos y lecciones	8 SP pts
Como...	instructor	
Quiero...	agregar módulos y lecciones con contenido y material	
Para...	estructurar el contenido del curso	

Criterios de aceptación:

- ✓ Un módulo tiene título y orden; contiene múltiples lecciones
- ✓ Una lección tiene título, descripción, URL de material y duración
- ✓ El instructor puede agregar, editar y eliminar módulos y lecciones
- ✓ El instructor solo gestiona módulos de sus propios cursos

Tareas técnicas:

Capa	Tarea	Responsable	Est.
BE	Endpoints CRUD de módulos: POST/PUT/DELETE /cursos/:id/modulos/:mid	Erick	3h
BE	Endpoints CRUD de lecciones anidadas en módulos	Erick	3h
FE	Vista "Gestionar módulos" en TeacherDashboard	Johan	4h
FE	Formulario de lección con campo URL de material	Johan	2h

US-07	Ver catálogo y detalle de curso	5 SP pts
Como...	alumno inscrito	
Quiero...	ver mis cursos con progreso y acceder al detalle	
Para...	navegar por los módulos y lecciones	

Criterios de aceptación:

- ✓ CatalogPage muestra solo cursos en los que el alumno está inscrito
- ✓ Cada tarjeta muestra título, descripción y barra de progreso real
- ✓ Si no hay cursos inscritos, se muestra botón para explorar
- ✓ CoursePage muestra módulos y lecciones con navegación anterior/siguiente

Tareas técnicas:

Capa	Tarea	Responsable	Est.
FE	CatalogPage conectada a GET /inscripciones/mis-cursos	Johan	2h
FE	CoursePage conectada a GET /cursos/:id con módulos reales	Johan	3h
FE	Sidebar con acordeón de módulos y estado de lección completada	Johan	2h

EP-03 — Evaluaciones

ID	Historia	Criterios clave	Resp.	SP
US-08	Realizar un quiz	Alumno ve preguntas sin respuestas correctas · Envía respuestas · Recibe calificación automática sobre 10 · Registro guardado en QuizResult	Erick + Johan	8 SP
US-09	Ver resultado del quiz	Pantalla de resultado muestra calificación, aprobado/reprobado y feedback · Historial visible en Mi Progreso	Johan	5 SP
US-10	Crear quiz (instructor)	Instructor crea preguntas de opción múltiple · Marca respuesta correcta · Quiz asociado a un curso específico	Erick	8 SP

EP-04 — Inscripciones y Progreso

ID	Historia	Criterios clave	Resp.	SP
US-11	Inscribirse a un curso	Alumno ve cursos disponibles en SearchPage · Botón Inscribirse crea registro · Email de confirmación · Curso aparece en CatalogPage	Erick + Johan	8 SP
US-12	Admin inscribe a alumno	Admin selecciona alumno y curso desde su panel · POST /inscripciones/admin · Actualización inmediata de la lista	Erick + Johan	3 SP
US-13	Progreso por lección	Botón "Completar" envía POST /completar · Porcentaje recalculado automáticamente · Persiste entre sesiones	Erick + Johan	5 SP
US-14	Dashboard de progreso	Alumno ve todos sus cursos con porcentaje · Historial de quizzes con calificaciones · Estadísticas generales	Johan	5 SP

EP-05 — Gestión Administrativa

ID	Historia	Criterios clave	Resp.	SP
US-15	CRUD de usuarios (admin)	Admin crea usuarios con rol definido · Activa/desactiva usuarios · Elimina usuarios · Modal de creación con validaciones	Erick + Johan	8 SP
US-16	Reportes de progreso	Admin ve tabla con alumnos, cursos en progreso, quizzes y tasa de aprobación · Datos reales de MongoDB	Erick + Johan	5 SP
US-17	Gestión de tareas (instructor)	Instructor crea tareas por curso (solo los suyos) · Fecha de entrega y puntos · Alumno entrega con comentario · Instructor ve quién entregó	Erick + Johan	8 SP

EP-06 — Notificaciones e Integración

ID	Historia	Criterios clave	Resp.	SP
US-18	Email de bienvenida	Al registrarse, el alumno recibe email con HTML personalizado · Sin bloquear la respuesta de la API · Configurado con Gmail SMTP	Erick	5 SP
US-19	Email de inscripción	Al inscribirse a un curso, el alumno recibe confirmación por email · Incluye nombre del curso y link a la plataforma	Erick	3 SP
US-20	Preparación Cloudinary	Variable CLOUDINARY_URL en .env · Campo materialURL en lecciones listo para recibir URLs · Documentación de integración pendiente	Erick	5 SP

4. Planificación de Sprints

Cada Sprint tiene una duración de 1 semana. La velocidad del equipo es de 32 SP por sprint.
Total del MVP: 131 Story Points distribuidos en 5 sprints.

Sprint 1 — Semanas 1-2			34 SP
Configuración del repositorio, base de datos, autenticación JWT y estructura base del proyecto			Story Points
US	Historia / Tarea	Responsable	Esfuerzo
US-01	Registro de alumno con JWT	Erick + Johan	5 SP
US-02	Inicio de sesión y roles	Erick + Johan	5 SP
US-03	Cierre de sesión	Johan	2 SP
US-04	Control de acceso por rol	Erick + Johan	5 SP
—	Configuración GitHub: ramas, protección, Projects	Anyelo	3 SP
—	MongoDB Atlas: cluster, seed data, índices	Anyelo + Erick	5 SP
—	Setup React + Vite, AuthContext, React Router	Johan	5 SP
—	Figma: pantallas de login, registro y catálogo	Alan	4 SP

Sprint 2 — Semanas 3-4			34 SP
API REST completa, CRUD de cursos, catálogo para el alumno y dashboard del instructor			Story Points
US	Historia / Tarea	Responsable	Esfuerzo
US-05	Crear y editar cursos (instructor)	Erick + Johan	8 SP
US-06	Módulos y lecciones con materiales	Erick + Johan	8 SP
US-07	Catálogo y detalle de curso (alumno)	Johan	5 SP
US-08	Quiz con calificación automática	Erick + Johan	8 SP
—	Diagrama de arquitectura y diagrama de secuencia	Anyelo + Alan	3 SP
—	Colección Postman exportada	Erick	2 SP

Sprint 3 — Semanas 5-6*Inscripciones a cursos, progreso, panel de administración y tareas del instructor***34 SP**

Story Points

US	Historia / Tarea	Responsable	Esfuerzo
US-09	Resultado y historial de quiz	Johan	5 SP
US-11	Inscripción a cursos (alumno)	Erick + Johan	8 SP
US-12	Admin inscribe a alumno	Erick + Johan	3 SP
US-13	Progreso por lección	Erick + Johan	5 SP
US-14	Dashboard de progreso del alumno	Johan	5 SP
US-17	Gestión de tareas por instructor	Erick + Johan	8 SP

Sprint 4 — Semanas 7-8*Panel de admin completo, reportes, notificaciones por email e integración de APIs externas***21 SP**

Story Points

US	Historia / Tarea	Responsable	Esfuerzo
US-15	CRUD de usuarios desde panel admin	Erick + Johan	8 SP
US-16	Reportes de progreso para admin	Erick + Johan	5 SP
US-18	Email de bienvenida con Nodemailer + Gmail	Erick	5 SP
US-19	Email de confirmación de inscripción	Erick	3 SP

5. Resumen de Esfuerzo por Integrante

Integrante	Rol	Épicas principales	Story Points	% del MVP
Alberto Anyelo	Scrum Master / DevOps	Gestión, BD, GitHub, CI/CD, Diagramas	22 SP	17%
Erick Hernandez	Backend Developer	EP-01, EP-02, EP-03, EP-04, EP-05, EP-06	55 SP	42%
Johan Anuart	Frontend Developer	EP-01, EP-02, EP-03, EP-04, EP-05	42 SP	32%
Alan Hazel	UX/UI Designer	Diseño Figma, QA visual, Documentación	12 SP	9%
TOTAL MVP			131 SP	100%

Escala de Story Points utilizada

Puntos	Complejidad	Descripción
1 SP	Trivial	Cambio de una línea, corrección de texto o configuración mínima
2 SP	Muy simple	Tarea de menos de 1 hora con impacto menor
3 SP	Simple	Tarea de 2-3 horas, sin dependencias complejas
5 SP	Moderada	Funcionalidad de medio día, puede tener dependencias
8 SP	Compleja	Funcionalidad de 1 día completo, múltiples capas involucradas
13 SP	Muy compleja	Funcionalidad de 2+ días, requiere coordinación entre integrantes

Definition of Done (DoD)

Una historia de usuario se considera DONE cuando:

- ✓ El código está en una rama feature/ con commits en formato Conventional Commits
- ✓ El Pull Request fue revisado y aprobado por el Scrum Master (Alberto)
- ✓ El código fue mergeado a la rama development sin conflictos
- ✓ El endpoint o pantalla fue probado manualmente y funciona según los criterios de aceptación
- ✓ No quedan credenciales expuestas en el código (variables en .env)
- ✓ El Issue correspondiente fue movido a "Hecho" en el tablero GitHub Projects